

PHASE 1 · WEEKS 1–3 · RAG FOUNDATIONS

Problem Statement

Build a single-source RAG assistant — embeddings, FAISS, re-ranking, and RAGAS evaluation, end to end.

Reference project: github.com/waseemkhan606/Single-Source-Retrieval

AI Engineering Masters Program

Topics This Project Must Demonstrate

LLM basics · prompt engineering · embeddings · FAISS vector storage · LCEL / RAG chains · query routing & decomposition · re-ranking · RAGAS evaluation.

Reference Project Summary

The reference build is a single-document RAG assistant: a user uploads one PDF, asks questions by text or voice, and receives answers grounded in that document with cited source chunks. The stack is a Next.js frontend, a FastAPI + LangChain backend, local **sentence-transformers** embeddings, a FAISS vector store, cross-encoder re-ranking, and RAGAS-based evaluation logged on every query.

The Assignment

Build your own single/few-source RAG assistant that is **not** a copy of the reference project's domain, but **is** architecturally equivalent: embeddings → vector retrieval → LCEL RAG chain → re-ranking → RAGAS evaluation, implemented end to end over a domain and document type you choose below.

Choose Your Domain — Pick One

- 1. Legal Clause Explainer** — upload a contract/ToS PDF; answer plain-English questions about clauses, obligations, and risk, with page citations.
- 2. Lecture Notes Q&A** — upload a set of course PDFs/slides; students query concepts and get answers grounded in the correct lecture.
- 3. Recipe & Nutrition Assistant** — upload cookbooks/nutrition PDFs; answer ingredient-substitution and nutrition questions with citations.
- 4. Financial Filings Analyst** — upload an annual report / 10-K; answer questions on revenue, risk factors, and guidance.
- 5. Medical Reference Assistant (non-diagnostic)** — upload drug leaflets or clinical guideline PDFs; answer factual dosage/interaction lookups only.
- 6. HR Policy Bot** — upload employee handbooks; answer leave, benefits, and conduct-policy questions.
- 7. Research Paper Digest** — upload arXiv-style papers; answer method/results/limitation questions with section citations.
- 8. Government Scheme Navigator** — upload public scheme PDFs; answer eligibility and application-process questions.
- 9. Real Estate Document Assistant** — upload lease/sale agreements; answer clause and obligation questions.
- 10. Open Topic** — student-proposed domain, subject to instructor approval before build starts.

Required Deliverables

- Working RAG pipeline: PDF upload → chunking → local embeddings → FAISS index → LCEL retrieval chain → LLM answer generation.
- Re-ranking stage: cross-encoder (or equivalent) re-ranking of retrieved chunks before generation.
- Query handling: basic routing/decomposition for multi-part questions — not a single naive retrieve-and-stuff.
- RAGAS evaluation: faithfulness, answer relevancy, and context precision logged per query (JSONL or equivalent).
- Source attribution in the UI/output: every answer shows which chunk(s)/page(s) it came from.
- README matching the reference project's documentation depth: setup, architecture diagram, tech stack table, API reference.
- A hosted, working link — see the Submission Guidelines document for exact requirements.

Out of Scope for This Phase

Multi-source hybrid search, GraphRAG, and Text-to-SQL belong to Phase 2. Multi-agent orchestration belongs to Phase 3. Production infrastructure — observability, caching, fallback routing — belongs to Phase 4. Keep this build scoped to single-source RAG.

How This Will Be Evaluated

- Does retrieval actually change with different questions — not hardcoded answers?
- Is re-ranking demonstrably improving result order (show a before/after comparison in the README)?
- Are RAGAS scores present and plausible — not all 1.0, and not missing?
- Is every citation-to-source claim verifiable by opening the cited page?